

AI224: Operating Systems — Lecture 9

Dr. Karim Lounis

Jan - May 2025



Memory Management

Memory Management

Memory Management

Definition

Memory Management System. Is a module of the operating system responsible for managing and controlling the use of the central memory.

The central memory or RAM, is a volatile type of memory circuit that is used to store information (e.g., text, programs, videos, images, etc).



The functions of the memory manager are:

- Keep track of the status of each portion of the central memory.
- Define and adopt a policy for allocating memory space to programs.
- Define and adopt a policy for releasing memory space.
- Protect the access to the memory space.
- Ensure the rearrangement (de-fragmentation) of free memory space.

Hardware to Improve Access

Because the CPU is very fast compared to the central memory, hardware is used to regulate this speed mismatch:

- CPUs have an internal fast memory, called cache memory, to store the most frequently and recently accessed instructions and data.

e.g., Think of instructions and data used within a loop

- When a particular data is required, the hardware first checks whether that data is in the cache or not, if not the data is copied from RAM to cache under the assumption of pretty-soon reuse.
- The update of a particular information must be reflected in all other caches that contain that information (in multiprocessor systems).

This is known as cache-coherency

- Sometimes, hardware is used to establish cache-coherency when DMA is used (fully cache coherent system).

Hardware to Improve Control

The OS alone is powerless, without hardware, it cannot enforce rules (e.g., a malicious process could bypass software checks).

- It is not the task of the OS to check which process is trying to perform illegal memory access (for performance reason).
- A hardware is used to implement this control.

E.g., Check addresses w-r-t to base and limit registers

- Each process has a dedicated memory space which defines the range of its legal addresses.

A dedicated hardware determines whether a given address is within that legal range or not

Yet, the hardware alone is dumb, it needs the OS to tell it what to protect.

Hardware to Improve Control

Per-process memory space protection can be implemented using two registers, namely, base and limit registers:

- **Base Register.** Holds the smallest legal physical address of a given process.
- **Limit Register.** Holds the size of the address space.

The CPU hardware compares every address generated by the CPU with these two registers:

1. Checks whether an $\text{address} \geq \text{base}$ or not. If not a trap is raised.
2. Checks whether an $\text{address} < \text{base} + \text{limit}$ or not. If not a trap is raised.

Consider $\text{base} = [0xF010]$ and $\text{limit} = [0x0190]$. Is the address $[0xF1A0]$ a legal address?

Coding, Compiling, Linking, Loading, and Executing

Compiling and loading user programs

Any user program goes through multiple steps before being loaded and executed:

- 1 The source code is written in a programming language (e.g., `main.c`).
`$gcc -c main.c`
- 2 The code is compiled into an object file by the compiler (e.g., `main.o`).
object files contain machine code (`$objdump -d main.o`)
- 3 The object file as well as other object files (static libraries) are bundled together into one executable file by a program called `Linker`.
e.g., `a.out`, `main.exe`, `main.elf`
- 4 The executable is loaded into the RAM for execution by the Loader.
- 5 During program execution, some modules are loaded into the memory and some libraries are linked to the program upon request.
i.e., dynamic loading (Loader) and dynamic linking (Linker)

Compiling and loading user programs

It is necessary for a program to have its entire code and data loaded into the central memory for its execution. However, the size of a program will be limited by the amount of space available in the memory.

Questions:

- Can we partially load programs and start running them?
- Can we load a program at diff memory locations during its execution?
- Does a program permanently resides in some memory location?
- Can we safely unload parts of a program while it is being executed?
- Can a program try to access some code that is not in the memory?

Dynamic Loading & Linking

Definition

Dynamic loading. Is a software mechanism in which modules referred in a given program are loaded into the central memory only when being used.

- Modules could be libraries (*.so, *.dll), binaries, classes, plugins, ...
- Useful for infrequently occurring cases, e.g., error handling routines.
- It is the task of the user to design the program in such a way its uses this mechanism. The OS just provides the APIs for that.
- Once a program is done with a module, the module is unloaded from the central memory.

A program may use different functions or methods which body is located inside libraries. For a better memory-space utilization, the libraries that contain the bodies of these functions are not loaded prior program execution.

Dynamic Loading & Linking

Definition

Dynamic linking. Is a software mechanism that allows a program to link the body of a function to the program by loading the respective library into RAM at runtime only when that particular function is being invoked.

- Usually used for system libraries (routines).
- Specifically regards dynamic libraries (*.so and *.dll).
- A **stub** is used for each library routine reference.
- The stub is replaced by the address of the routine for execution.
- Some operating systems support only static linking (archive libraries).

Logical and Physical Addresses

Every byte in the memory has a specific address that may range from 0 to some maximum, known as **physical address**.

e.g., if the address size $M=32\text{bits}$ then addresses range from $0x00000000$ to $0xFFFFFFFF$

Whenever a program is brought into memory, it occupies a certain number of memory locations, and the set of all physical addresses used by that program is called **physical address space**.

e.g., from $0xCD18B235$ to $0xCD18C000$

During compilation, all variables and procedures in the source code of a program are assigned some specific addresses, known as **logical addresses**, where the set of these address is called **logical address space**.

i.e., in a machine code, all references to data and code are made by specifying the logical addresses and not by the variable or procedure names

Address Binding

Definition

Binding. Is the process of setting up some relationships between 2 entities.

Initially, there is no correspondence between a program identifier space and the memory address space. To execute a program, we need to map its program identifier space into memory address space.

Definition

Address Binding. Is a mapping from one address space to another.

It can take place at one of the following times:

- Compile time,
- Loader time,
- Execution time.

Address Binding

Assuming a 16-bit operating system that starts assigning logical addresses from 0x0000 to 0xFFFF (and 32 bytes for each function code, for simplicity):

Code	Logical Addressing Layout				Assembly
<pre>int A = 22; void empty() { int a = 10; return; } int main() { empty(); return A; }</pre>	Address Range	Section	Content	Assigned logical addresses	section .data A: 22 section .text empty:
	0x0000 - 0x00FF	Code	empty(), main()	empty — 0x0000 to 0x001F main() — 0x0020 to 0x003F	push bp
	0x0100 - 0x0103	Data	Global variable A=20 takes 4 bytes	A in 0x0100 and 0x0103	mov bp, sp
	0xFFFFB - 0xFFFFF	Stack	Local variable a=10 takes 4 bytes Local variable b=true take 1 byte	a in 0xFFFF to 0xFFFFC b in 0xFFFFB	sub sp, 5; mov [bp-4], 10 mov [bp-5], 1 mov sp, bp pop bp
					ret
					main:
					call empty
					mov ax, [A]
					ret
					.start:
					call main
					hlt

Address Binding (Compile Time)

- 1 **Compile time.** If it is known, at the compile time, which addresses the program will occupy in the main memory. In this case, logical addresses are the same as that of physical addresses [static allocation].

$$(\text{symbolic @} \xrightarrow{\text{compile}} \text{absolute @})$$

In this scheme a binary program is called **absolute binary image**.

The program should always be loaded in the same location.

Used in single-task operating systems e.g., *.com files in MS-DOS.

Address Binding

- ② **Load time.** If it is not known, at compile time, which addresses the program will occupy in the main memory, the compiler generates relocatable addresses that will be converted into absolute addresses at the load time.

$$(\text{symbolic @} \xrightarrow{\text{compile}} \text{relocatable @} \xrightarrow{\text{load}} \text{absolute @})$$

Used in multiprogramming systems where it may not be possible to know in advance where in the main memory a binary image will be loaded.

It depends on which partition is available [**dynamic allocation**].

In this scheme a binary program is called **relocatable binary image**.

Relocatable address r also called relative addresses (relative to $0x000$).

The relocatable loader updates all relocatable addresses depending on where in the main memory the image is loaded.

Address Binding

- 3 **Execution time.** If a process can be moved during its execution from one memory partition to another, then binding should be delayed till runtime [**dynamic relocation**].

(symbolic @ $\xrightarrow{\text{compile + loader}}$ relocatable @ $\xrightarrow{\text{execution}}$ absolute @)

When the CPU executes a program, it generates relative addresses and not physical memory addresses.

Those relative addresses are popularly known as logical addresses.

Logical addresses are translated at runtime by a dedicated hardware known as MMU (Memory Management Unit).

The Memory Management Unit can be implemented in different ways depending on how main memory is allocated.

Logical and Physical Address Space

Definition

Logical address: A.k.a, virtual address, is a memory address that is generated by the CPU (what the program sees) during program execution.

e.g., in `Mov Ax, [0xF23E]` the address `[0xF23E]` is a logical address

The user uses the logical address to access the physical address.

The set of all logical addresses generated by a program are called the program logical address space.

Definition

Physical address: is a memory address that is seen by the memory unit (the real address in RAM) that points to a physical memory location.

e.g., logical addresses are translated/mapped by the a hardware component called MMU (Memory Management Unit) into physical addresses

Swapping

Definition

Swapping: Is the process of moving a program (i.e., its address space) from the RAM to the hard disk drive (swap out) or the reverse (swap in).

It allows to;

- Virtually increase the size of the main memory.
- Increase the degree of multiprogramming.

It is performed by an operating system module called dispatcher.

- It has a cost: For a 100MB process to be moved to HDD at 50MBps, it takes around $2 + \lambda$ seconds.
- On modern OSs, swapping is triggered when the total of unused (available) memory falls below a threshold amount.
- Another approach consists of swapping a portion of a program and not the whole program.
- Not used in mobile OSs, where OSs terminates processes if memory is needed.

Memory Allocation

Memory Allocation Strategies

Memory Allocation

There are various strategies that are used to manage the memory. All these strategies allocate memory to processes using either:

- Contiguous memory allocation.

Single partition Vs multiple partitions.

Fixed size Vs Variable size.

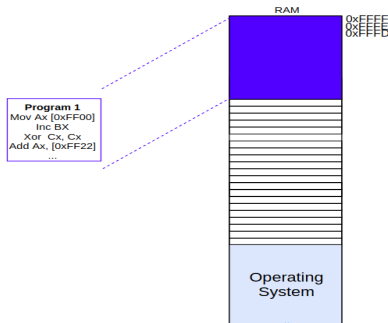
- Noncontiguous memory allocation.

Paging, Segmentation, and Segmentation with paging.

Single partition (One Contiguous Zone)

In this strategy, the main memory is divided into two zones, one to store the operating system's code and the other to store user programs.

- In single-task operating systems, only one program can be loaded at a time.
- Programs are limited by the size of the user space.
- The memory is not well exploited (used and shared).
- A λ -Byte program can occupy the whole user space.

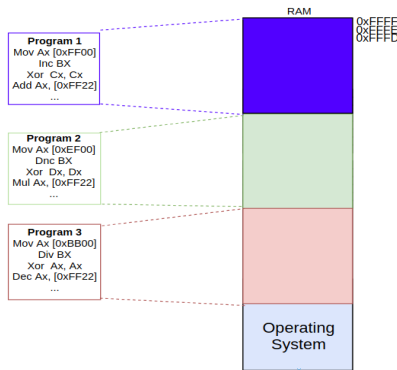


Multiple partitions

In this strategy, the main memory is divided into two zones, one to store the OS code and the other for multiple user programs.

Multiple Static Partitions. In this scheme, partitions have the same size (a system parameter). It was originally used by **IBM OS/360**.

- Used by multi-task OSs where multiple programs can be loaded at a time.
- The user zone is divided into multiple partitions of equal size.
- Degree of multiprogramming is bound by the # of partitions.
- Internal fragmentation.
- No longer used.

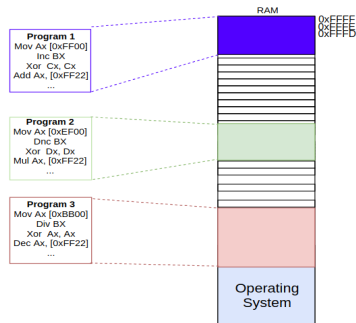


Multiple partitions.

In this strategy, the main memory is divided into two zones, one to store the OS code and the other for multiple user programs.

Multiple Variable Partitions. Partition have variable sizes (depending on program needs).

- Used by multi-task operating systems where multiple programs can be loaded at a time.
- The user zone is divided into multiple partitions of variable size.
- A base address and a size are used in PDT (Partition Description Table).
- External fragmentation.



Fragmentation

There are two types of fragmentation:

1 External fragmentation.

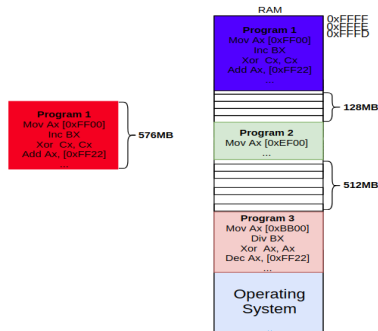
happens when there is a certain amount of non-contiguous memory space available but cannot be used by some programs.

Compaction.

Relocate programs to bundle the small memory holes into one large whole.

Segmentation and/or Paging.

program address space is not contiguous.



Fragmentation

There are two types of fragmentation:

② Internal fragmentation.

happens when a program is using a memory partition which size is larger than the program.

Segmentation and/or Paging.

program address space is not contiguous.



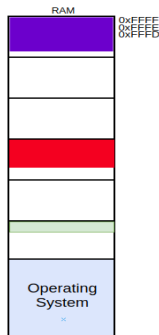
Program 1



Program 2



Program 3



Address Space Translation at Runtime

When address binding is performed at execution time (runtime), the program execution generates logical addresses (a.k.a., virtual addresses):

e.g., in Mov Ax, [0xF1254F74] the address [0xF1254F74] is a logical address

The set of all logical address that a process generates is called the program's **logical address space** (or **virtual address space**).

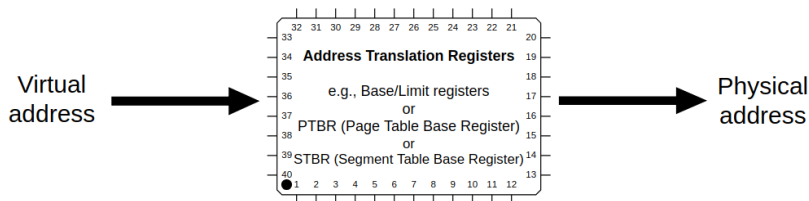
Different processes may have the same virtual address space.

e.g., Execute multiple instances of a given program

The OS needs the flexibility to assign different parts of main memory to various processes i.e., **each virtual address will be mapped (translated) into a final physical address.**

Address Translation at Runtime

During the execution of a program, the address translation takes place by the mean of a dedicated hardware:



There exists three methods for address translation:

- 1 Base/Limit registers.
- 2 Paging.
- 3 Segmentation.

Memory management

Base/Limit registers-based address translation

Base/Limit registers-based address translation

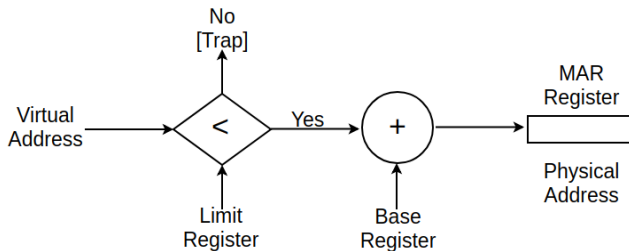
During compilation time, all program symbolic addresses are converted into virtual address:

Virtual addresses in range $[0 \dots \text{Limit}[$

These virtual addresses are mapped by the hardware into:

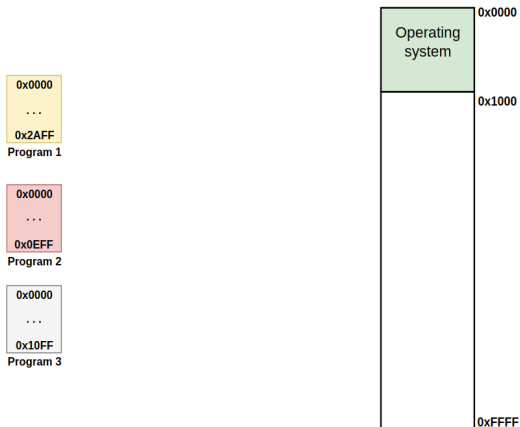
Physical addresses in range $[\text{Base} \dots \text{Base} + \text{Limit}[$

The hardware also performs a check to determine whether a given virtual address is a legal addresses or not.



Base/Limit registers-based address translation

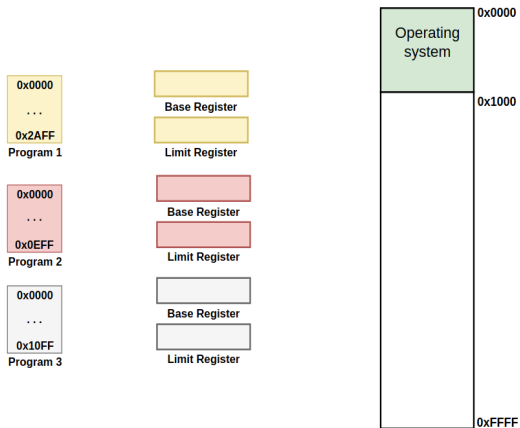
Consider 3 compiled programs Program 1, Program 2, and Program 3:



Each program has its own virtual address space. An address is on 2 Bytes.

Base/Limit registers-based address translation

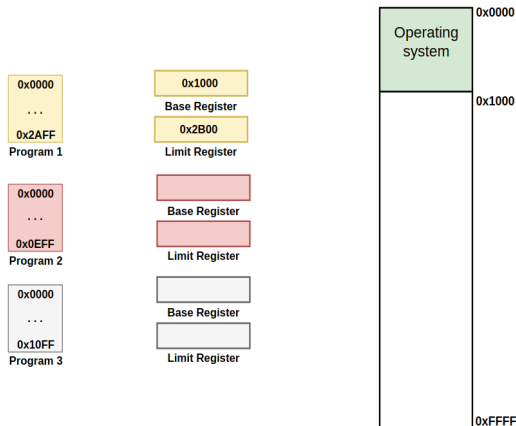
For each program, the OS assigns a base register value depending on the memory status:



Each program's limit register contains the size of its virtual address space.

Base/Limit registers-based address translation

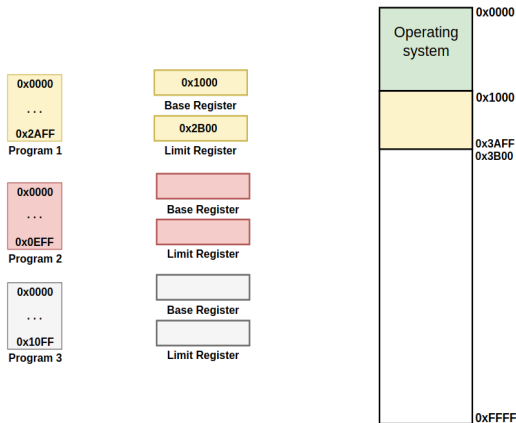
For example, the OS assigns the value of 0x1000 to the a base register of Program 1:



The limit register will be filled by the value 0x2B00.

Base/Limit registers-based address translation

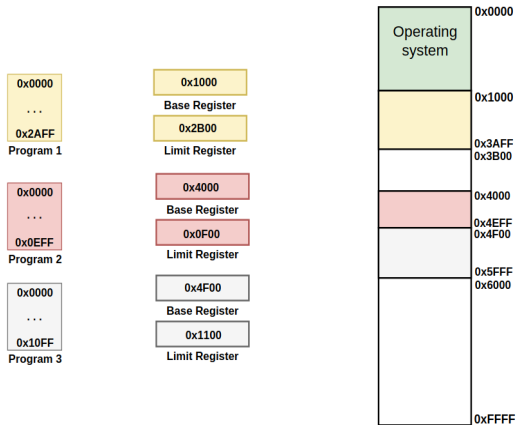
For example, the OS assigns the value of 0x1000 to the a base register of Program 1:



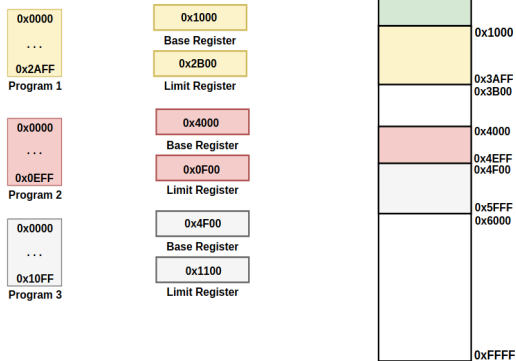
The limit register will be filled by the value 0x2B00.

Base/Limit registers-based address translation

The OS assigns the value of 0x4000 and 0x4F00 to the a base registers of Program 2 and Program 3 respectively:

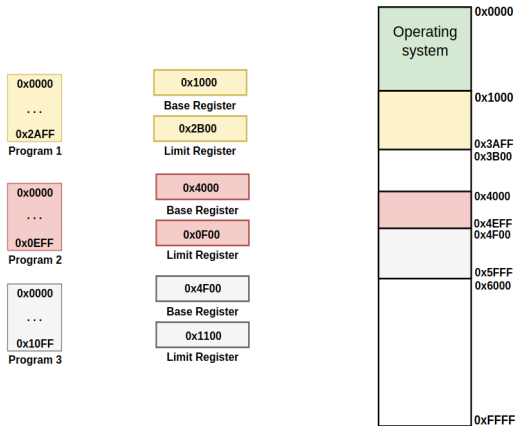


Base/Limit registers-based address translation



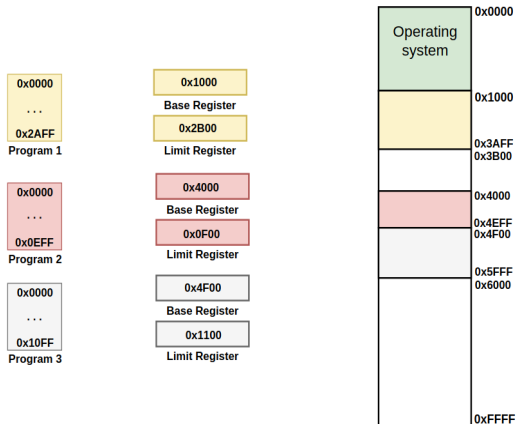
What would be the physical address for the virtual address 0x001A for each program?

Base/Limit registers-based address translation



The virtual address 0x001A gets translated to 0x101A for Program 1, 0x401A for Program 2, and 0x4F1A for Program 3.

Base/Limit registers-based address translation



However, this is used when variable sized partition allocation approach is adopted (i.e., Contiguous Memory Allocation) by the operating system.

Non-Contiguous Memory Allocation: Paging

Paging

Paging

Definition

Paging. Is a memory management scheme that allows programs to be stored in the main memory in a noncontiguous way by dividing their virtual address spaces into fixed and equal-sized blocks known as **pages**.

The physical memory is divided into fixed-sized blocks called **frames**.

Pages (of the program's virtual address space) have the exact same size as the **frames** (in the physical address space).

When a program is to be executed, its **pages** (actually, some of them) are loaded into available **frames** in the main memory.

Depending on the computer architecture, the size of a page is a power of 2, varying between 512 Bytes and 1GB per page.

On GNU/Linux, you can get the page size by typing the following command:

```
root@Human-Device:/# getconf PAGESIZE
4096
```


Paging-based address translation

Every virtual address generated by the CPU is divided into two parts:

page number | page offset

1. **Page number.** Denoted by p , is used as an index into a page table to identify a page entry.

Definition

Page table. Is a system data structure, associated to each process, indicating the base address for each page entry in the main memory.

2. **Page offset.** Denoted by d , is the displacement within the page.

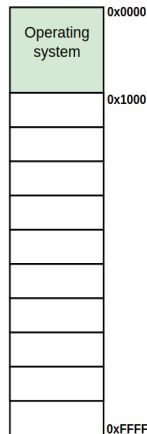
i.e., A page occupies multiple memory locations (bytes), the offset d allows to specify a particular physical location within the frame (one byte)

Paging-based address translation

Consider a program whose virtual address space is broken into 4 pages, and consider a memory organized into multiple frames:

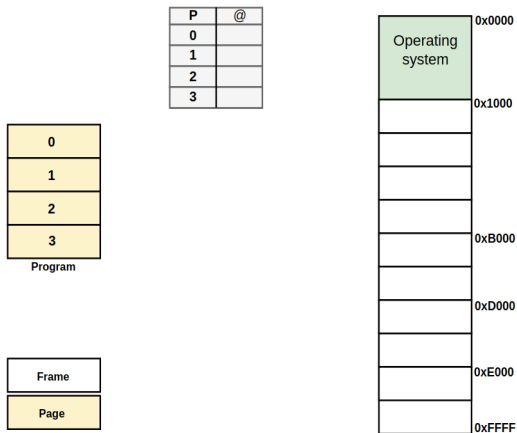


Program



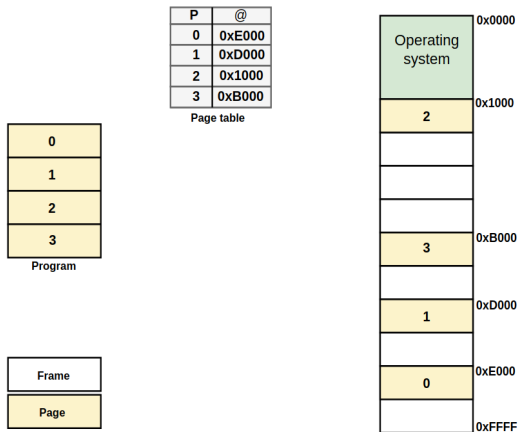
Paging-based address translation

Each page in the virtual space is identified by an index (page number), and for each process a page table is associated.



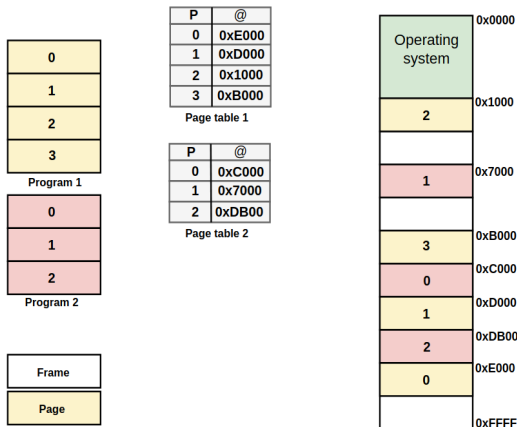
Paging-based address translation

Each page number is associated to a physical address in the main memory called page base address:



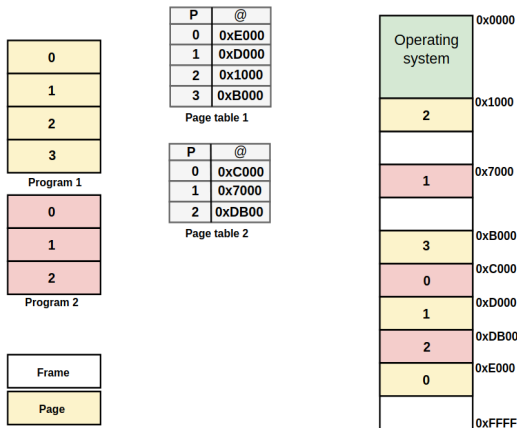
Paging-based address translation

Another program would have its own page table and frames in the memory:



Paging-based address translation

Another program would have its own page table and frames in the memory:



How do we generate the physical address of a specific location in memory when paging is used?

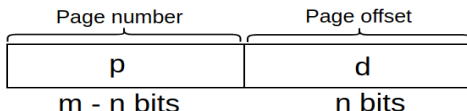
Paging-based address translation

The page size (like the frame size) is defined by the hardware. The size of the page is a power of 2, varying from 512 bytes and 1GB per page.

Usually, a page has a size of 4KB (\$ `getconf PAGESIZE`)

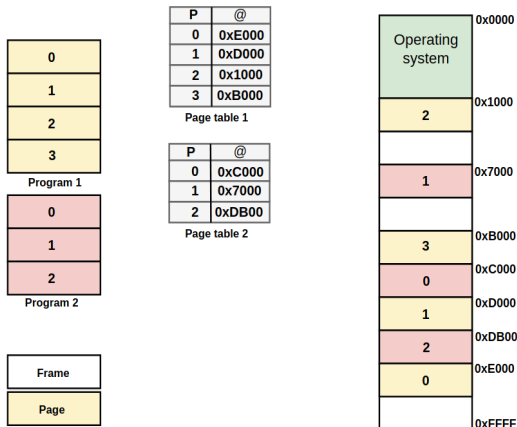
The use of a power of 2 as a page size makes the address translation easier.

If the size of the virtual address space is 2^m , and a page size is 2^n bytes, then the high-order $m - n$ bits of a virtual address designate the page number, and the n low-order bits designate the page offset.



Paging-based address translation

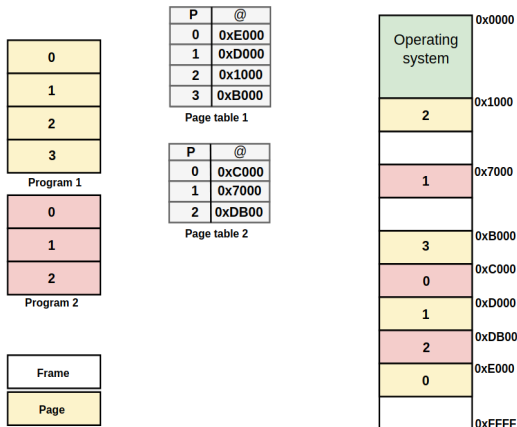
Example. Consider that the addresses are on 16 bits (i.e., $m = 16$).



If the size of a page is 8KB ($n = 13$), then the page number will be on p bits the offset will be on d bits.

Paging-based address translation

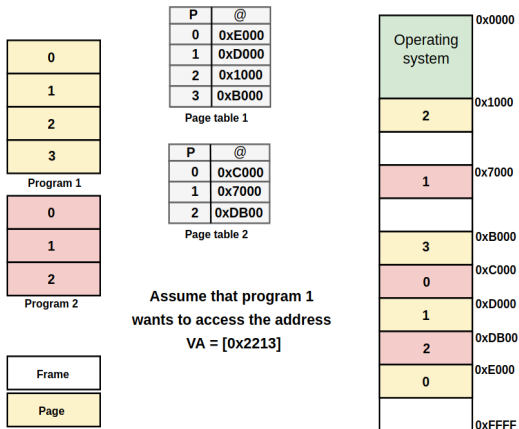
Example. Consider that the addresses are on 16 bits (i.e., $m = 16$).



If the size of a page is 8KB ($n = 13$), then the page number will be on **3 bits** the offset will be on **13 bits**.

Paging-based address translation

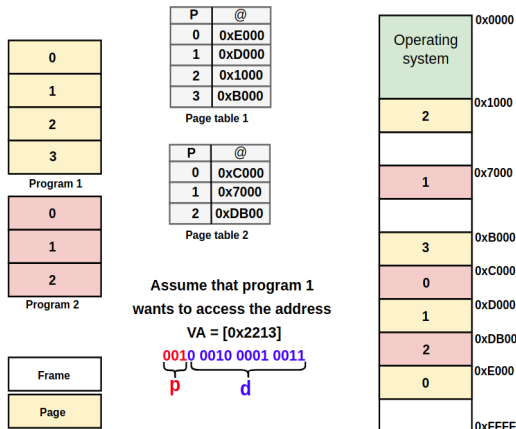
Example. Consider that the addresses are on 16 bits (i.e., $m = 16$).



If the size of a page is 8KB ($n = 13$), then the page number will be on **3 bits** the offset will be on **13 bits**.

Paging-based address translation

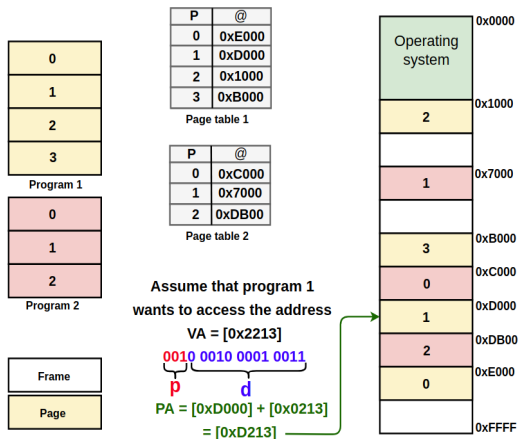
Example. Consider that the addresses are on 16 bits (i.e., $m = 16$).



If the size of a page is 8KB ($n = 13$), then the page number will be on **3 bits** the offset will be on **13 bits**.

Paging-based address translation

Example. Consider that the addresses are on 16 bits (i.e., $m = 16$).



If the size of a page is 8KB ($n = 13$), then the page number will be on **3 bits** the offset will be on **13 bits**.

Paging and fragmentation

When paging is used:

- There is no external fragmentation: as any available frame can be allocated to a process that needs it.
- There may be some internal fragmentation: since frames are allocated as units and a program memory requirement may not coincide with page boundaries, the last frame allocated may not be completely full.

e.g., if page size is 2KB, a process of 72,766 bytes will need 35 pages + 1086 bytes. The process will be allocated 36 frames with a 962 bytes of internal fragmentation (wasted space).

If a process is 71,681 bytes, what will be the amount of the internal fragmentation?

Paging and fragmentation

When paging is used:

- There is no external fragmentation: as any available frame can be allocated to a process that needs it.
- There may be some internal fragmentation: since frames are allocated as units and a program memory requirement may not coincide with page boundaries, the last frame allocated may not be completely full.

e.g., if page size is 2KB, a process of 72,766 bytes will need 35 pages + 1086 bytes. The process will be allocated 36 frames with a 962 bytes of internal fragmentation (wasted space).

If a process is 71,681 bytes, what will be the amount of the internal fragmentation?

Internal fragmentation is: 2047 bytes

- End.